



TECHNICAL REFERENCE

Power Glove Vision Configuration Reference

Active files, installed copies, fields, secrets, and generated state.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Use this reference to find a setting, change a game mapping, tune a gesture, or look up a command. Each section identifies the active file and explains what its values mean.

Use the [installation guide](#) for the initial deployment and pairing procedure. Return here when you need to change a host, camera, game, gesture threshold, or network setting.

KEEP THE TOKEN PRIVATE PowerGlove Vision uses one shared token to authenticate controller packets and profile changes. Never paste it into an issue, screenshot, command line, public backup, or Git commit.

The three places configuration lives

PowerGlove Vision runs on two computers. A repository template is not always the file the running system reads.

Location	What it controls	Preferred way to change it
UNO Q Setup page	Console address, controller port, startup profile, camera, and pairing token	Browser Setup page
UNO Q application files	Gesture sensitivity and advanced runtime defaults	Edit only when tuning is required
RetroPie <code>/etc/powerglove/</code>	UNO Q address, per-game profile selection, and receiver token	Protected files on RetroPie

The examples under the repository's `config/` directory are installation templates. Editing them does not change an already installed system. The active copies are identified in each section below.

In commands and examples, replace these placeholders:

Placeholder	Replace with
<code>UNO-Q-NAME.local</code>	Your UNO Q hostname or reserved IP address
<code>RETROPIE-NAME.local</code>	Your RetroPie hostname or reserved IP address
<code>/home/arduino/ArduinoApps/powerglove-vision</code>	Required directory for the supported UNO Q installer and shutdown helper; do not substitute a different path

Find the setting or command you need

Task	Section
Install and pair both machines	Installation Guide
Change the camera or startup profile	UNO Q settings
Repair pairing or token permissions	Pairing and token management
Change the UNO Q destination on RetroPie	RetroPie connection settings
Make a game select a profile	Register games and select profiles
Adjust gesture sensitivity	Tune gesture sensitivity
Understand an option or command	Command-line reference

UNO Q settings

Open the ordinary Setup page at:

```
http://UNO-Q-NAME.local:8088/setup
```

Pairing uses the secure Setup page instead:

```
https://UNO-Q-NAME.local:8443/setup
```

The secure page uses a locally generated certificate. During pairing, compare the certificate fingerprint shown by the browser with the identifier shown on the UNO Q matrix before entering the one-time PIN.

Settings shown in the browser

Setting	Default	Meaning and recommendation
Console hostname or IP	Empty (not configured)	Set your RetroPie hostname (<code>RETROPIE-NAME.local</code> in examples) or a reserved LAN address and pair through Connection before starting controls. Learn and local settings work without a destination. Existing saved destinations are preserved.
Controller port	55355	UNO Q to RetroPie controller-state port. Leave it at the default unless both ends are changed.
Startup profile	bad_street_brawler	Profile used before a registered game selects another one.
Tracking aid	none	none, white, or black. In the current release this is an informational diagnostic label; it does not change MediaPipe tracking.
Camera	auto	Prefer auto. Use a number from 0 through 99 only when automatic selection chooses the wrong capture device.
Generate a new token	Off	Rotates the shared secret. This immediately breaks the existing pairing until RetroPie is paired again.

Selecting **Save** validates the fields, writes them atomically with private permissions, and restarts the vision worker using the saved calibration. Recalibrate only if you have moved the camera, changed your playing position, or notice unwanted movement while your hand is at rest.

The Dashboard profile selector changes only the current active profile. It does not rewrite `device.json` or change the Setup page's startup profile. RetroPie may replace a Dashboard selection when a game starts or ends.

The first time you activate a profile, the app opens the camera and loads the gesture tracker. This can take longer than subsequent profile changes. Dashboard and Learn show **Starting camera and gesture tracking** with elapsed time until vision is active. The **Calibrate** button is disabled until initialization finishes. If **Gestures off** is selected at startup, the camera stays closed until you select an active profile or open Learn.

Learn, calibration, and live readings

Learn starts the camera even when **Gestures off** is selected and uses Program H for practice. It pauses controller delivery and restores the selected profile when you leave. With several Learn tabs open, practice remains active until the last tab closes or its lease expires. A six-second lease timeout handles an unexpected browser close. Loading Dashboard also clears a stale session; reload Learn if you want to begin practice again.

The eleven lessons include A (index curl), B (thumb curl), GLOVE ZAP (forward push), Start, and Select. Completing every lesson earns Glove Master; skipped lessons must be revisited. **Start again** clears session progress. The practice indicators do not change a game's gesture mapping.

Reading or control	Meaning
Finger curl	Learn shows values from 0 to 1; Dashboard uses a compact 0-to-3 display. Ordinary curl actions engage at 0.50 and release below 0.35.
V sign	Index and middle curl must be below 0.28; ring and little curl must exceed 0.42. Hold for about 0.7 seconds to send Start.
Thumbs-up	Thumb curl must be below 0.32 and all four finger curls above 0.42. Hold for about 0.7 seconds to send Select.

Reading or control	Meaning
Live hand measurements	Shows curl values, thresholds, enlarged landmarks, and forward movement relative to the calibrated hand size.
Calibrate	Replaces the saved resting reference. The button turns red while sampling, then blue with a brief completion message.
<code>inference_ms</code> and <code>send_ms</code>	Tracking calculation and local send time; neither measures the full delay from camera movement to game response.

Finger recognition uses the strongest joint bend, including the base knuckle; a middle-knuckle bend alone can qualify. Thumb recognition uses the stronger of its two outer joints. MediaPipe 3D world landmarks are preferred. The fallback uses normalized depth with image aspect-ratio correction; palm movement still uses image coordinates. The legacy Arduino landmark bridge keeps its 2D interpretation because its depth units differ.

Learn uses the same held finger state as gameplay. Push remains recognized until movement falls below its release threshold, and a confirmed menu pose still satisfies its lesson after the short controller pulse ends. The browser preview is capped at 15 fps; status updates follow each tracking calculation.

The app reuses its saved resting reference across Learn, gameplay, profile changes, and restarts. It calibrates automatically only when that reference is missing or invalid. Use **Calibrate** after moving the camera or changing your playing position. Keep your palm near the resting position when practising finger curls so unintended movement does not obscure the finger readings. See [Saved neutral-hand calibration](#) for storage and recovery details.

Start controller and **Stop controller** affect live delivery only. After an application or system restart, delivery remains stopped until you start it.

Active UNO Q device file

The Setup page maintains this private file inside the application:

```
/home/arduino/ArduinoApps/powerglove-vision/data/device.json
```

A typical device configuration file contains the following fields:

```
JSON
{
  "receiver": "RETROPIE-NAME.local",
  "port": 55355,
  "token": "private-random-value-created-by-the-application",
  "profile": "bad_street_brawler",
  "glove_color": "none",
  "camera": "auto"
}
```

Use the Setup page for routine changes. If you must edit the JSON directly, stop the application first, keep the token unchanged, validate the file, and restore mode `0600`. JSON does not allow comments or trailing commas.

```
SH
python3 -m json.tool data/device.json >/dev/null
chmod 0600 data/device.json
```

Deleting `device.json` causes PowerGlove Vision to create a new token and first-run defaults. You must then pair RetroPie again.

Camera selection

`auto` searches stable `/dev/v4l/by-id/` capture-device links first, ignores known codec-only video nodes, and then considers ordinary `/dev/video*` capture devices. This is the most reliable choice when USB enumeration changes after a reboot.

Use an explicit camera number only for troubleshooting. For example, `0` selects `/dev/video0`, but Linux may assign that number to a different device after hardware is reconnected. Keep the camera on a powered USB hub when the UNO Q cannot supply stable power by itself.

Supported startup profiles

The valid startup profile identifiers are:

```
off
bad_street_brawler
super_glove_ball
program_a program_b program_c program_d program_e
program_f program_g program_h program_i
```

The startup profile does not assign a profile to a ROM. Per-game selection is controlled by the RetroPie game registry described below.

Pairing and token management

Both machines must hold the same token:

Machine	Active private file
UNO Q	Application <code>data/device.json</code> , in the <code>token</code> field
RetroPie	<code>/etc/powerglove/token</code>

Use one-time-code pairing whenever possible:

```
SH
sudo /opt/powerglove/bin/powerglove-pair
```

Leave that command running on RetroPie, then complete pairing at <https://UNO-Q-NAME.local:8443/setup>. The code is single use and expires after two minutes. Password pairing is also available when RetroPie accepts SSH password login; the password is used for one encrypted operation and is not stored.

The RetroPie token must contain at least 16 characters and should remain owned by `root`, readable by the `input` group, and inaccessible to other users:

```
SH
sudo chown root:input /etc/powerglove/token
sudo chmod 0640 /etc/powerglove/token
```

Recover without browser pairing

Use this fallback only when neither browser pairing method works. Both machines must already have the software installed.

- 1 In App Lab, open the active application's private `data/device.json` and locate its `token` value.
- 2 On RetroPie, run `sudo nano /etc/powerglove/token`. Replace the file contents with that same value on one line, without quotation marks. Do not enter it as a shell command.
- 3 Save with Ctrl+O, confirm the filename, and exit with Ctrl+X. Apply the ownership and permission commands above.
- 4 Run `sudo systemctl restart powerglove-receiver.service`, then test controller delivery from Dashboard. Clear the token from your clipboard and close the private file afterward.

If you generate a new token in Setup, pair the devices again immediately. Do not transfer the new token through a command-line argument; process listings and shell history can expose it.

RetroPie connection settings

The active launcher file is:

```
/etc/powerglove/launcher.json
```

It tells the runcommand hooks where to send profile changes when a game starts or exits.

```
JSON
{
  "uno_q": "UNO-Q-NAME.local",
  "port": 55356,
  "token_file": "/etc/powerglove/token",
  "registry": "/etc/powerglove/games.json",
  "timeout": 0.4
}
```

Field	Meaning
<code>uno_q</code>	UNO Q hostname or reserved address reachable from RetroPie.
<code>port</code>	RetroPie to UNO Q profile-control port. This is <code>55356</code> , not the controller-state port.
<code>token_file</code>	Protected shared-token file. Keep the token out of this JSON file.
<code>registry</code>	Active ROM-to-profile mapping.
<code>timeout</code>	Seconds to wait for each acknowledgement. The hook retries up to three times and never prevents a game from launching.

Validate changes before launching a game:

```
SH
python3 -m json.tool /etc/powerglove/launcher.json >/dev/null
```

Use a `.local` hostname when multicast DNS is reliable on your LAN. A reserved DHCP address is a useful fallback. Do not use an address that may later be assigned to another device.

Register games and select profiles

Edit mappings in Games

Games

Edit the game mappings installed on your RetroPie. Saving affects the next game launch.

Use the exact ROM filename, including its extension: `Joust (USA).7z` and `Joust (USA).nes` need separate entries. Matching ignores letter case. Only NES and Famicom launches use these mappings.

► Available profile identifiers

Game mappings JSON

```
{
  "games": {
    "Bad Street Brawler (USA).nes": "bad_street_brawler",
    "Bad Street Brawler (USA).zip": "bad_street_brawler",
    "Bad Street Brawler (USA).7z": "bad_street_brawler",
    "Super Glove Ball (USA).nes": "super_glove_ball",
    "Super Glove Ball (USA).zip": "super_glove_ball",
    "Super Glove Ball (USA).7z": "super_glove_ball",
    "Joust (USA).nes": "program_b",
    "Gyruss (USA).nes": "program_c",
    "Defender II (USA).nes": "program_e",
    "Sesame Street 1-2-3 (USA).nes": "program_f",
    "Gun.Smoke (USA).nes": "program_g",
    "Knight Rider (USA).nes": "program_i",
    "Joust (USA).zip": "program_b",
    "Joust (USA).7z": "program_b",
    "Gyruss (USA).zip": "program_c",
    "Gyruss (USA).7z": "program_c",
    "Defender II (USA).zip": "program_e",
    "Defender II (USA).7z": "program_e",
    "Sesame Street 1-2-3 (USA).zip": "program_f",
    "Sesame Street 1-2-3 (USA).7z": "program_f",
    "Gun.Smoke (USA).zip": "program_g"
  }
}
```

ValidateFormatSaveReloadDownload backupRestore previous save

Installed mappings loaded.

Games section below pairing on Setup

Games is a section of **Setup**, below pairing; it is not a separate navigation tab.

- 1 Open **Setup** -> **Games** in the UNO Q website. Both machines must be online and paired. The page reads the registry used by the installed RetroPie launch hook.
- 2 Select **Download backup** to keep a copy of the last verified installed registry on your computer.
- 3 Edit the JSON, adding the exact ROM filename and a supported profile identifier inside `games`. Expand **Available profile identifiers** for the choices. Preserve your existing entries.
- 4 Select **Validate**. It checks JSON syntax, supported profiles, and duplicate filenames, including names that differ only by letter case. **Format** tidies the JSON without saving it.
- 5 Select **Save**. Wait for confirmation that RetroPie saved the file and the UNO Q read it back successfully.

- 6 Launch or restart the game and confirm its profile on Dashboard.

Saving does not change the current game's profile. **Restore previous save** swaps in the last valid version. **Reload** discards your draft after confirmation. If another editor changed the installed registry, saving is refused; download or copy your draft before reloading. Connection failures leave the draft in the browser. Leaving or refreshing the page can discard unsaved work.

The Games section needs the [powerglove-games.service](#) installed by the current RetroPie setup workflow. If it reports an unavailable service, update the RetroPie installation, check pairing, and ensure TCP [55358](#) is reachable from the UNO Q. Games does not require an SSH password after pairing.

Registry format

The active game registry is:

```
/etc/powerglove/games.json
```

PowerGlove Vision matches the exact ROM basename, including its extension, without regard to letter case. Directory names are ignored. Automatic profile selection currently applies only when RetroPie reports the system as [nes](#) or [famicom](#); other systems turn gesture control off.

```
JSON
{
  "games": {
    "Bad Street Brawler (USA).nes": "bad_street_brawler",
    "Super Glove Ball (USA).zip": "super_glove_ball",
    "Joust (USA).nes": "program_b"
  }
}
```

Add the exact filename shown in EmulationStation. Zipped and 7-Zip copies need their own entries because [.nes](#), [.zip](#), and [.7z](#) are different basenames.

Included game	Profile
Bad Street Brawler	bad_street_brawler
Super Glove Ball	super_glove_ball
Joust	program_b
Gyruss	program_c
Defender II	program_e
Sesame Street 1-2-3	program_f
Gun Smoke	program_g
Knight Rider	program_i

The table above is the complete set of games recognized automatically by the shipped registry. Programs A, D, and H are fully implemented profiles rather than omitted games: [program_a](#) is a pinball control scheme, [program_d](#) reverses all four directions for challenge or accessibility use, and [program_h](#) provides general-purpose movement with pulsed buttons. They deliberately have no default ROM assignment.

Any appropriate NES or Famicom ROM can use one of those profiles after you add its exact basename to the [games](#) object. Every profile value must be one of the supported identifiers; an unknown value invalidates the registry.

```
SH
sudo python3 -m json.tool /etc/powerglove/games.json >/dev/null
```

Test profile communication independently of a game:

```
SH
sudo /opt/powerglove/bin/powerglove-profile \
  --uno-q UNO-Q-NAME.local \
  --token-file /etc/powerglove/token \
  --profile program_b
```

Use `--profile off` to stop gesture output. A successful request prints an acknowledgement and starts the matrix glove attract animation. In this healthy idle state the camera and MediaPipe tracker are closed, while the website and authenticated profile listener remain available.

Check a queued profile change

The launch helper reports **profile queued** when the UNO Q has authenticated and queued the request. Dashboard shows the applied profile and ROM name once the worker processes it. Camera startup can take longer than the acknowledgement; an acknowledgement does not mean that the camera is ready or that controller delivery is enabled.

App Lab's [local:profile_control](#) brick publishes UDP [55356](#) through a small relay to the worker. The relay holds no pairing token and forwards signed packets unchanged. Both this brick and [local:avahi_resolver](#) must be present in [app.yaml](#), so the services return when App Lab regenerates its containers. The setup and Wi-Fi update helpers also add their includes to existing Compose configuration. On the UNO Q, check the published port with:

```
SH
docker port powerglove-vision-profile-relay-1 55356/udp
```

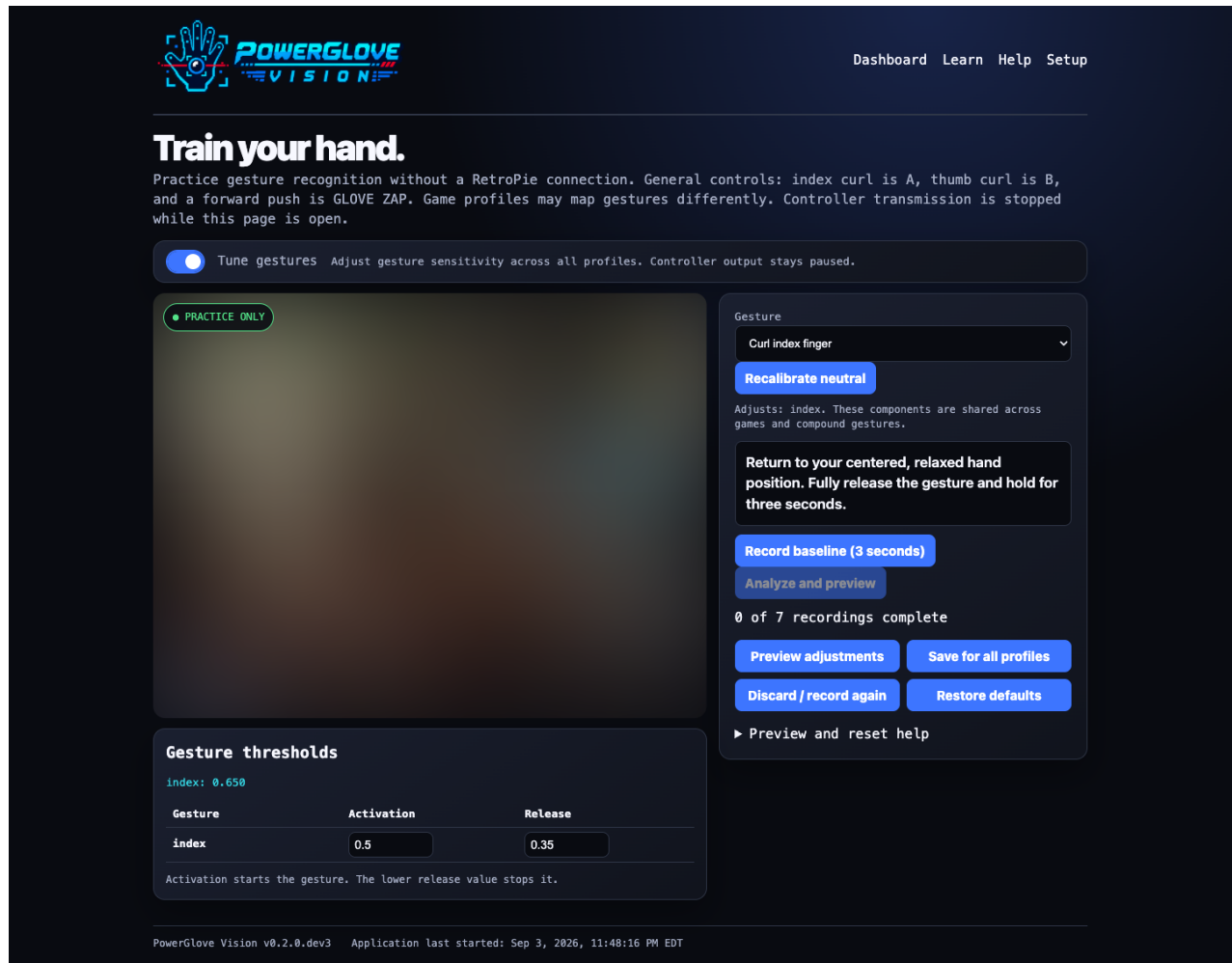
Expect a host binding for port [55356](#). If it is missing, update the application and rerun UNO Q setup. On RetroPie, compare the game's actual filename, including [.nes](#), [.zip](#), or [.7z](#), with [/etc/powerglove/games.json](#). These are separate exact entries. Updating the template does not overwrite an installed registry; add missing names while preserving your custom mappings.

Tune gesture sensitivity

Use **Learn -> Tune gestures** to adjust sensitivity. You do not need to edit [config/profiles.json](#); it remains the supplied defaults for each profile.

- 1 Show your whole hand in the camera and wait for tracking. Calibrate your comfortable resting position if necessary.
- 2 Switch on **Tune gestures** and select a gesture. Instructions and action buttons stay beside the camera; the Activation and Release table sits beneath the camera. Directions, finger curls, wrist rolls, push/pull, and compound gestures are available.
- 3 Hold your centered, relaxed hand still and select **Record baseline**. Each recording lasts three seconds.
- 4 Follow the prompt to perform and hold the gesture, then record its release. Repeat this pair three times: seven recordings including the baseline.

- 5 Select **Analyze and preview**. The app uses clear, fresh camera measurements to suggest activation and release values. If resting and performed measurements overlap, repeat the recordings with a clearer gesture and a complete release.
- 6 Try the temporary preview. You may adjust the numeric values and select **Preview adjustments** before deciding whether to save.
- 7 Select **Save for all profiles** to keep the adjustment. **Discard / record again** removes unsaved changes. **Restore defaults** removes saved adjustments for the selected gesture's components.



Tune mode with the Activation and Release table beneath the blurred camera

The camera imagery is blurred in this reference screenshot. On a wide screen, instructions and buttons sit beside the camera; on a narrow screen, they appear first. The matrix shows **T** while tuning and **L** in ordinary practice.

Activation is the point where a gesture begins; release is the lower point where it stops. Separate values prevent rapid on/off flickering. Directions and fingers can be adjusted independently. Compound gestures share component thresholds, so changing a finger also affects other gestures that use it. Suggested menu-pose adjustments tune the closed fingers; already extended fingers retain their existing settings unless you adjust them manually. Button assignments and menu hold timing remain unchanged.

Only adjusted components override all game profiles. Untuned components retain their profile's supplied values. Personal adjustments are saved atomically in [data/gesture-tuning.json](#) and survive application restarts and normal updates. No images or recordings are saved. The versioned format is:

```
JSON
{
  "version": 1,
  "thresholds": {
    "index": {"on": 0.6, "off": 0.4}
  }
}
```

Each pair must contain finite numbers with $0 \leq \text{off} < \text{on}$. Finger and pull activation cannot exceed 1; wrist rotation cannot exceed 2; movement and push cannot exceed 4. These are normalized measurements, not distances in centimetres.

Tuning pauses controller delivery. A game launch may update the selected game but cannot interrupt tuning or send game input. Leaving Tune discards its preview; a disconnected browser's session expires after six seconds. Return to Dashboard and explicitly start controller delivery when ready to play. **Recalibrate neutral** changes the resting reference separately and invalidates any current recordings.

Supplied profile defaults

The following fields describe the shipped `config/profiles.json`. They remain useful for understanding the defaults; personal tuning is managed through Learn.

Threshold fields

Field	What it measures	Effect of lowering the value
<code>move_on</code>	Palm displacement from center, normalized by palm size	Movement activates sooner
<code>move_off</code>	Palm displacement at which active movement releases	Movement stays active farther back toward center
<code>curl_on</code>	Normalized finger curl, where <code>0</code> is straight and <code>1</code> is tightly curled	Curl actions activate with less bend
<code>curl_off</code>	Curl amount at which an active curl releases	Curl stays active until the finger is straighter
<code>roll_on</code>	Wrist rotation from the centred angle	Roll actions activate with less rotation
<code>roll_off</code>	Rotation at which active roll releases	Roll stays active closer to neutral
<code>push_on</code>	Relative increase in apparent hand size from center	Push actions activate with less forward movement
<code>push_off</code>	Depth change at which an active push releases	Push stays active closer to the centred depth
<code>pulse_hz</code>	Repetition rate for profiles that pulse an action	Repeated actions become slower
<code>loss_release_ms</code>	Tracking-loss delay before all controls release	Controls release sooner after the hand disappears

For each gesture, keep the `_off` value lower than its `_on` value. The gap is hysteresis: it prevents a value near the activation point from rapidly turning on and off. A very large gap can make the control feel sticky.

The supplied defaults are:

```
JSON
{
  "move_on": 0.38,
  "move_off": 0.24,
  "curl_on": 0.50,
  "curl_off": 0.35,
  "roll_on": 0.58,
  "roll_off": 0.40,
  "push_on": 0.34,
  "push_off": 0.18,
  "pulse_hz": 7.0,
  "loss_release_ms": 120
}
```

The `super_glove_ball` profile uses movement activation and release thresholds of `0.32` and `0.20`, making movement more responsive. Its wrist-roll thresholds are `0.70` and `0.50`, its pulse rate is `8.0` Hz, and its push thresholds are slightly higher than the defaults. Profiles A through I use `program_defaults` unless an exact profile object such as `program_b` is added.

When adjusting numeric values in Tune, change one pair at a time in steps of approximately `0.02` to `0.05`, then test from the same camera position. Useful adjustments include:

- Lower `move_on` if directional movement requires too much travel.
- Raise `move_off` if a direction remains held after returning toward center.
- Raise an `_on` value when an action triggers unintentionally.
- Increase `pulse_hz` when a repeating action is too slow.
- Keep `loss_release_ms` short enough to release safely but long enough to tolerate a few missed camera frames.

Saved personal tuning applies without reopening the camera. The saved neutral calibration is reused; recalibrate only if your physical setup has changed or your resting hand position produces unwanted movement. Gesture-to-button assignments are implemented by each profile in the application; threshold changes adjust sensitivity but do not remap buttons.

RetroPie receiver and virtual controller

The receiver verifies authenticated UDP packets and creates a Linux `uinput` gamepad named `PowerGlove Vision`. Its installed service is:

```
/etc/systemd/system/powerglove-receiver.service
```

The supplied service listens on all local interfaces at UDP port `55355`, reads `/etc/powerglove/token`, and releases held controls when a socket receive times out after 250 milliseconds. This is a receive timeout, rather than a separate timer for the last valid packet. If you change the controller port in UNO Q Setup, add the same `--port` value to the service's `ExecStart`, then reload and restart:

```
SH
sudo systemctl daemon-reload
sudo systemctl restart powerglove-receiver.service
```

The companion `powerglove-receiver.timer` starts the receiver 45 seconds after boot. This lets EmulationStation finish its initial controller scan first. Keep the timer enabled and the service itself disabled for boot activation. Starting the

receiver too early can cause frontend pauses and conflicts with other USB devices, such as a BitPixel display.

```
SH
systemctl is-enabled powerglove-receiver.timer
systemctl is-enabled powerglove-receiver.service
sudo systemctl status powerglove-receiver.service
sudo journalctl -u powerglove-receiver.service -n 100 --no-pager
```

The virtual gamepad appears only after the first authenticated controller packet. On the UNO Q, select **Start controller** and show a centred hand before deciding that the device is missing.

RetroArch autoconfiguration

Install the supplied mapping at:

```
/opt/retroPie/configs/all/retroarch/autoconfig/PowerGlove_Vision.cfg
```

It matches the virtual device name plus vendor and product IDs **1:1**, uses the **udev** input driver, and maps D-pad directions, A, B, Start, Select, and four axes to a standard RetroPad. It does not configure an I-PAC, 8BitDo controller, or any other physical controller.

If RetroArch has a hand-written override for this device, remove or reconcile that override before diagnosing the supplied autoconfiguration.

UNO Q shutdown helper

The standard installation includes the host shutdown helper so Dashboard and Setup can halt Linux cleanly. It consists of two systemd units and one boot-time readiness rule:

Repository file	Installed path	Purpose
uno-q/powerglove-system-shutdown.path	/etc/systemd/system/powerglove-system-shutdown.path	Watches for one fixed shutdown request in the application's private data directory
uno-q/powerglove-system-shutdown.service	/etc/systemd/system/powerglove-system-shutdown.service	Removes that request and asks systemd to halt Linux cleanly
uno-q/powerglove-system-shutdown.conf	/etc/tmpfiles.d/powerglove-system-shutdown.conf	Recreates the unprivileged readiness marker at boot or after application replacement

Install them with [scripts/install-uno-q-shutdown-helper.sh](#). The installer also creates the private [data/.shutdown-enabled](#) marker that allows the web UI to offer the action. The tmpfiles rule restores that marker at boot. The application cannot use this mechanism to execute an arbitrary privileged command; it can only create the fixed request after an explicit confirmation.

Do not change the request path in only one component. The web application, path unit, service, and marker must continue to agree. After installation, confirm that [powerglove-system-shutdown.path](#) is enabled and active.

Network ports and trust boundary

Keep PowerGlove Vision on a trusted home or cabinet LAN. Do not forward these ports through a router or expose them directly to the Internet.

Port	Direction	Purpose
UDP 55355	UNO Q to RetroPie	Authenticated live controller state
UDP 55356	RetroPie to UNO Q	Authenticated game-profile requests and acknowledgements
TCP 8088	Browser to UNO Q	Dashboard, Help, Learn, and ordinary Setup UI, including Games
TCP 8443	Browser to UNO Q	TLS Setup and pairing workflow
TCP 55358	UNO Q to RetroPie	Paired game registry reads, saves, and restoration
TCP 55357	UNO Q to RetroPie	Temporary one-time-code pairing helper

The two UDP ports serve different purposes despite their similar numbers. The `port` in UNO Q `device.json` is normally 55355; the `port` in RetroPie `launcher.json` is normally 55356.

The ordinary dashboard is reachable by other devices on the LAN. Pairing and shutdown require additional protections, but the project assumes that the LAN itself is trusted. Review [SECURITY.md](#) before using a shared, guest, school, or public network.

UNO Q matrix artwork

The built-in display is a 13-by-8 monochrome blue LED matrix with eight brightness levels. Treat it as a very small dot-matrix display in the spirit of a pinball DMD or monochrome BitPixel display, rather than as a miniature screen. The sketch encodes status, pairing, profile identifiers, and the gestures-idle Power Glove attract sequence.

Matrix artwork should use broad motion, recognizable silhouettes, and strong separation between the subject and its brightest effect. A moving spark needs a dim body underneath it, a medium halo, and a full-bright core. Pulses should emphasize an outline instead of illuminating every pixel at maximum brightness, which erases the shape on the physical display. Transitional objects need to cross several columns and remain visible for more than one frame; isolated one-pixel changes are easily lost to persistence and viewing angle.

Always judge animation timing and grayscale on the physical UNO Q. A source grid or browser mock-up is useful for finding malformed frames, but it cannot reproduce LED bloom, exposure, or perceived persistence. A short video covering several complete loops is the preferred review artifact for later refinements.

Files most users should not edit

File or directory	Purpose
<code>app.yaml</code>	Arduino App Lab application metadata and exposed ports
<code>sketch/sketch.yaml</code>	UNO Q sketch platform and pinned Arduino library dependencies
<code>pyproject.toml</code>	Python package metadata, supported interpreter range, and optional dependencies
<code>python/worker-wheels/</code>	Platform-specific MediaPipe worker dependency supplied by the App Lab installation ZIP
<code>data/models/hand_landmarker.task</code>	Checksum-verified model downloaded when vision is first activated
<code>data/uv-cache/</code> and <code>data/uv-python/</code>	Generated private worker runtime and package cache
<code>.cache/app-compose.yaml</code>	App Lab generated container configuration
<code>data/.shutdown-enabled</code>	Readiness marker installed by the fixed-purpose shutdown helper included in standard setup
<code>output/pdf/</code>	Generated PDF editions; public editions are served by Help, while the cabinet quick reference remains private

Changing manifests can prevent App Lab from starting the application. Generated files and downloaded runtime assets should not be committed or added to an App Lab installation ZIP. The installation ZIP is built as:

```
output/app-lab/PowerGlove-Vision-Uno-Q.zip
```

The installation ZIP intentionally excludes private `data/`, downloaded models, caches, tests, Git metadata, and the cabinet-specific quick-reference PDF. It includes only the nine allowlisted public PDF editions used by Help. When an active profile first needs vision, the application downloads the pinned Google Hand Landmarker model and verifies its SHA-256 checksum. The model stays unopened while **Gestures off** is selected.

Automated quality and package verification

Maintainers use `.github/workflows/quality.yml` for pull requests, pushes to `main` and `dev`, and manual runs. The workflow tests the project on supported Python versions, checks Python and shell syntax, validates JSON, audits source and documentation, rebuilds and inspects the PDF set, and verifies the App Lab installation ZIP. The workflow has read-only repository permissions and uses no deployment credentials.

Ordinary installers do not need to edit this workflow. If you fork the project, keep live UNO Q credentials and cabinet deployment out of hosted CI; deployment should remain an explicit action from a trusted machine.

Back up and update safely

Back up custom configuration before replacing an installation:

Item	Why it matters
UNO Q <code>data/calibration.json</code>	Preserves your neutral hand position, size, and wrist angle; recalibrate if the physical setup changes

Item	Why it matters
UNO Q <code>data/device.json</code>	Contains device settings and the private token
UNO Q <code>config/profiles.json</code>	Contains any custom sensitivity values
RetroPie <code>/etc/powerglove/games.json</code>	Contains local ROM mappings
RetroPie <code>/etc/powerglove/launcher.json</code>	Contains local host and path settings
RetroPie <code>/etc/powerglove/token</code>	Contains the matching private token
RetroArch <code>PowerGlove_Vision.cfg</code>	Contains any deliberate local mapping changes

Store token-bearing backups privately with restricted permissions. The supplied Wi-Fi deployment script preserves the UNO Q `data/` directory. The App Lab installation ZIP never contains your token or downloaded model.

After an update, confirm that the active files under `/etc/powerglove/` still contain your local hostnames and ROM names. Updating repository templates does not automatically migrate active configuration.

Troubleshooting by symptom

Symptom	Configuration checks
Dashboard works but no virtual controller appears	Start the controller, show a centred hand, verify the receiver service and shared token, then check UDP <code>55355</code> .
Controller appears but a game uses the wrong gestures	Confirm the system is <code>nes</code> or <code>famicom</code> and the exact ROM basename exists in <code>/etc/powerglove/games.json</code> .
Game launches slowly while UNO Q is offline	Confirm <code>timeout</code> remains near <code>0.4</code> ; the hook retries but must never block game launch indefinitely.
Profile command is not acknowledged	Check the UNO Q name, UDP <code>55356</code> , pairing token, and the UNO Q application status.
Gestures off shows a blinking X	Update PowerGlove Vision; Gestures off should show the glove attract animation and must not open the camera.
Camera disappears after reboot	Return Camera to <code>auto</code> , check powered-hub and cable stability, and inspect the UNO Q dashboard error.
Movement triggers too late	Center again first; if repeatable, lower <code>move_on</code> slightly for the active profile.
Direction remains stuck	Raise <code>move_off</code> slightly, keep it below <code>move_on</code> , and verify tracking-loss release.
Pairing suddenly fails after a Setup change	A rotated token invalidates the old pairing; run the pairing flow again.
EmulationStation pauses or another USB device behaves unexpectedly at boot	Verify receiver startup is controlled by the 45-second timer and the service is not independently enabled at boot.

Configuration file catalog

Repository file	Active or installed copy	Used by
config/device.example.json	UNO Q application data/device.json	Vision supervisor and Setup UI
config/profiles.json	UNO Q application config/profiles.json	Gesture engine
config/games.json	RetroPie /etc/powerglove/games.json	Launch hook and profile selector
config/launcher.example.json	RetroPie /etc/powerglove/launcher.json	RetroPie launch and exit hooks
retropie/retroarch/PowerGlove Vision.cfg	RetroArch autoconfig directory	RetroArch input system
retropie/powerglove-receiver.service	/etc/systemd/system/	Privileged virtual-controller receiver
retropie/powerglove-games.service	/etc/systemd/system/	Paired Games editor service on RetroPie
data/gesture-tuning.json (runtime only)	UNO Q application data/gesture-tuning.json	Global personal threshold overlays
retropie/powerglove-receiver.timer	/etc/systemd/system/	Delayed boot activation
uno-q/powerglove-system-shutdown.path	/etc/systemd/system/	Fixed shutdown request watcher
uno-q/powerglove-system-shutdown.service	/etc/systemd/system/	Fixed clean-shutdown action
uno-q/powerglove-system-shutdown.conf	/etc/tmpfiles.d/	Boot-time shutdown readiness marker
.github/workflows/quality.yml	GitHub Actions	Automated tests and release verification
app.yaml	UNO Q application root	App Lab
sketch/sketch.yaml	UNO Q application sketch directory	Arduino build system
pyproject.toml	Repository or deployed application root	Python packaging and tests

When a change goes wrong, restore the last known-good active file rather than copying every repository template over the installation. Validate JSON, restart only the affected service or application, and test packet delivery before changing controller mappings.

Local hostname resolution inside App Lab

If the console name fails, use **Test console name** in Connection, then follow [hostname troubleshooting](#). A router-reserved IPv4 address is a fallback, not a setup requirement.

Both machine installers install [avahi-daemon](#) and [libnss-mdns](#) and enable Avahi at boot. The UNO Q host dependency supports native hostname lookups; it does not replace the app-owned resolver used inside the container. Setup check mode verifies the dependency and Avahi service, and the UNO Q check tests the configured destination inside the app.

The app-owned [local:avahi_resolver](#) brick survives App Lab container regeneration. It mounts [/run/avahi-daemon](#) read-only in a separate unprivileged service and exposes only IPv4 [.local](#) queries through [data/.avahi-resolver.sock](#). It has no network interface or published port. The app prefers this private socket; the direct host socket remains a compatibility fallback. Both sockets are runtime files, not configuration to back up or distribute.

All gameplay and pairing lookups use this resolver. Answers expire after five seconds, so DHCP changes do not require editing an address. Ordinary DNS names use the system resolver. Generic container [getent](#) is not the app's mDNS test; use Connection's hostname test or the setup command's check mode.

Known limitation: UNO Q restarts after Shutdown

On the tested UNO Q, a graceful [halt](#) still leads to an automatic restart, including when connected directly to a Mac without the powered hub. The helper requests halt correctly, but remaining stopped is not verified. Do not use loss of the website or a fixed delay as confirmation that power can safely be removed. See the installation guide for the investigation status and Arduino guidance.

Saved neutral-hand calibration

The worker saves its completed neutral reference in [data/calibration.json](#). It includes palm position, apparent size, and wrist angle; it is not a personally trained gesture model. Learn, gameplay, profile changes, camera reconnects, and worker restarts reuse this reference. **Calibrate** explicitly replaces it after sampling completes; an interrupted calibration preserves the previous saved reference. Recalibrate after moving your camera or changing your seating position.

On first use, or if the saved file is missing or invalid, the worker samples an initial reference automatically. Hold your hand in a comfortable neutral position, then use **Calibrate** if necessary. A storage failure is reported as [calibration_save_error](#) in status; the reference remains usable in memory but will not survive a worker restart. The file is local runtime data, not a source or release-package file.

Command-line reference

Use this section to look up a flag without interrupting the installation steps. It covers every project command and script, plus the external-command options used in these guides. Defaults describe this source version, not a guarantee about a future release. For external tools' other options, use their own help or manual pages.

An **option** begins with `-` or `--`. A **positional argument** is a value supplied in a particular place, such as [retropie](#) after [setup-machine.py](#). Replace example hostnames and file paths with your own. Square brackets in usage descriptions

mean optional; do not type them. Options that take a value need both the option and its value, such as `--port 55355`.

Install or inspect a machine

Run `sudo python3 scripts/setup-machine.py MACHINE [OPTIONS]` from the project directory on the target Linux machine. This is the recommended installer for both UNO Q and RetroPie. `--check` performs read-only checks; using `sudo` also lets those checks read protected token files.

Argument or flag	Default	Meaning
<code>MACHINE</code>	Required	<code>retropie</code> or <code>uno-q</code> ; selects the machine to install or inspect.
<code>--peer HOST</code>	None	Required for a new RetroPie launcher configuration; supplies the UNO Q hostname or IPv4 address. Existing launcher settings are preserved. On UNO Q, it prints guidance but does not change the saved receiver address.
<code>--check</code>	Off	Checks the existing installation without installing, restarting, or changing it.
<code>-h, --help</code>	-	Prints usage and exits.

Exit codes are `0` for success, `1` for an installation/check failure, and `2` for outstanding user action. Argument errors also use argparse's exit code `2`. The current check always asks for human gameplay confirmation.

Run the RetroPie receiver

Use `/opt/powerglove/bin/powerglove-receiver` on RetroPie. Normal operation is managed by its systemd service. A manual receiver cannot share the same port with that service: stop the service before a manual diagnostic run, then restart it afterward. Use `--token-file` rather than placing a token in shell history.

Flag	Default	Meaning
<code>--listen ADDRESS</code>	<code>127.0.0.1</code>	Local address to bind. <code>0.0.0.0</code> accepts packets on all local IPv4 interfaces, as the installed service requires.
<code>--port NUMBER</code>	<code>55355</code>	UDP port for controller packets; must match UNO Q settings.
<code>--token VALUE</code>	None	Supplies the shared token directly. Use only as an advanced alternative; the value can appear in process arguments.
<code>--token-file PATH</code>	None	Reads the shared token from a protected file. Supply exactly one of this flag and <code>--token</code> . The token must contain at least 16 characters.
<code>--timeout-ms NUMBER</code>	<code>250</code>	Socket receive timeout in milliseconds; a timeout releases held controls. Use a positive value.
<code>--dry-run</code>	Off	Prints received controls instead of creating a virtual input device.
<code>-h, --help</code>	-	Prints usage and exits.

Example: inspect packets without sending input to Linux. Run each command on RetroPie and press Ctrl+C to end the diagnostic receiver before restarting the service.

```
SH
sudo systemctl stop powerglove-receiver.service
sudo /opt/powerglove/bin/powerglove-receiver --listen 0.0.0.0 --token-file /etc/powerglove/token --dr
sudo systemctl start powerglove-receiver.service
```

Start one-time-code pairing

Use `/opt/powerglove/bin/powerglove-pair` on RetroPie with `sudo`. It opens a temporary TLS server, prints a code, installs the received token, and restarts the receiver. Complete the browser steps while it is running.

Flag	Default	Meaning
<code>--listen ADDRESS</code>	<code>0.0.0.0</code>	Local IPv4 address for the temporary server.
<code>--port NUMBER</code>	<code>55357</code>	Pairing server TCP port. The browser pairing client uses the standard port; keep the default for that workflow.
<code>--token-file PATH</code>	<code>/etc/powerglove/token</code>	Destination for the paired token; keep it aligned with the receiver's token file.
<code>--timeout SECONDS</code>	<code>120</code>	Lifetime of the pairing server. Use a positive value; the code is single use and attempts are limited.
<code>--receiver-service NAME</code>	<code>powerglove-receiver.service</code>	systemd service to restart after pairing succeeds.
<code>-h, --help</code>	-	Prints usage and exits.

The command returns `0` after pairing completes or `2` when the pairing window times out. Other failures are reported as errors.

Select a profile from RetroPie

Use `/opt/powerglove/bin/powerglove-profile` to test profile selection. It sends an authenticated request to the UNO Q. This changes the active profile; it does not change the saved startup profile or turn controller delivery on.

Flag	Default	Meaning
<code>--uno-q HOST</code>	Required	UNO Q hostname or IPv4 address.
<code>--port NUMBER</code>	<code>55356</code>	UNO Q profile-control UDP port.
<code>--token VALUE</code>	None	Supplies the token directly; prefer the protected-file option.
<code>--token-file PATH</code>	None	Reads the token from a file. Supply exactly one token option.
<code>--registry PATH</code>	<code>/etc/powerglove/games.json</code>	Registry used when <code>--profile</code> is omitted.
<code>--system NAME</code>	<code>nes</code>	System used for registry selection and request metadata. Automatic selection accepts <code>nes</code> or <code>famicom</code> .
<code>--rom PATH</code>	Manual selection	ROM path or filename used for registry lookup and displayed game metadata.
<code>--profile NAME</code>	None	Overrides registry lookup. Accepts <code>program_a</code> through <code>program_i</code> , <code>bad_street_brawler</code> , <code>super_glove_ball</code> , or <code>off</code> .

Flag	Default	Meaning
<code>--timeout SECONDS</code>	<code>0.4</code>	Wait for each acknowledgement; the sender makes up to three attempts.
<code>-h, --help</code>	-	Prints usage and exits.

Exit codes are `0` for acceptance, `2` for a timeout, and `3` for rejection. Acceptance means the request is queued; confirm the applied profile on Dashboard. Malformed arguments and file errors can also stop the command.

Run the paired Games service

The RetroPie installer starts this service automatically. Its installed command is `powerglove-games`; developers can also run `python3 -m powerglove_vision.game_registry`.

Flag	Default	Purpose
<code>--settings</code>	<code>/etc/powerglove/launcher.json</code>	Read the administrator-configured registry and token-file locations.
<code>--listen</code>	<code>0.0.0.0</code>	Bind address for the paired LAN service.
<code>--port</code>	<code>55358</code>	TCP port. The UNO browser proxy uses the standard port.
<code>--help</code>	Not applicable	Print usage without starting the service.

The service uses a single request handler with a three-second socket timeout, 64-KiB registry limit, and 64 pending challenges. Challenges expire after fifteen seconds and can be used once. The previous registry is saved beside the active file as `games.json.previous` (or the configured filename plus `.previous`). Installer-generated systemd permissions allow writing to the configured registry directory. Rerun installation if an administrator changes that directory.

Forward game-launch events

The installer connects `powerglove-retropie-hook` to RetroPie automatically. Use its wrapper scripts from existing runcommand hooks; do not run them again as an extra installation step.

Argument or flag	Default	Meaning
<code>ACTION</code>	Required	<code>start</code> selects a registered game's profile; <code>end</code> turns gestures off.
<code>SYSTEM</code>	Empty	Optional first metadata argument, such as <code>nes</code> .
<code>EMULATOR</code>	Empty	Optional second metadata argument supplied by RetroPie; accepted for compatibility.
<code>ROM</code>	Empty	Optional third metadata argument; its filename selects the profile.
<code>COMMAND</code>	Empty	Optional fourth metadata argument; accepted for compatibility. Quote it as one argument.
<code>--settings PATH</code>	<code>/etc/powerglove/launcher.json</code>	Reads destination, token path, registry path, and timeout settings.
<code>-h, --help</code>	-	Prints usage and exits.

`retropie/runcommand-onstart-powerglove.sh` forwards RetroPie's four positional arguments. `retropie/runcommand-onend-powerglove.sh` needs none. Neither wrapper has its own flags. The executables in `retropie/bin/` forward their arguments to the corresponding Python commands.

Run the standalone vision tracker

`powerglove-vision` is the advanced camera worker, not the full App Lab website. Install the package's `vision` dependencies in a compatible Python environment before using it. Normal UNO Q use should start through App Lab instead.

Flag	Default	Meaning
<code>--receiver HOST</code>	Required	RetroPie hostname or IPv4 address. An empty string permits local-only tracking but cannot deliver controls.
<code>--port NUMBER</code>	55355	Destination UDP controller port.
<code>--token VALUE</code>	Required	Shared token. This worker currently has no <code>--token-file</code> flag; its token appears in process arguments.
<code>--profile NAME</code>	<code>bad_street_brawler</code>	Initial profile, one of the eleven supported profiles or <code>off</code> .
<code>--camera VALUE</code>	<code>auto</code>	Camera selection; use <code>auto</code> or a camera index.
<code>--width PIXELS</code>	640	Requested capture width; the camera may negotiate another size.
<code>--height PIXELS</code>	480	Requested capture height.
<code>--fps NUMBER</code>	30	Requested capture rate; not a guarantee of tracking or game frame rate.
<code>--glove-color VALUE</code>	<code>none</code>	<code>none</code> , <code>white</code> , or <code>black</code> ; an informational label, not a different recognition model.
<code>--no-mirror</code>	Off	Disables horizontal image mirroring.
<code>--config PATH</code>	Project <code>config/profiles.json</code> , if present	Alternative gesture-threshold file. Otherwise built-in defaults are used.
<code>--model PATH</code>	Default downloaded model	Alternative MediaPipe model. The standard <code>hand_landmarker.task</code> filename uses the verified model workflow.
<code>--web-host ADDRESS</code>	0.0.0.0	Address for the worker's diagnostic web server.
<code>--web-port NUMBER</code>	8088	Worker diagnostic port. App Lab overrides this to 8089 on loopback behind its main web server.
<code>--no-matrix</code>	Off	Disables direct matrix integration. App Lab uses this because its supervisor controls the matrix.
<code>--profile-listen ADDRESS</code>	0.0.0.0	Local IPv4 address for authenticated profile requests.
<code>--profile-port NUMBER</code>	55356	UDP port for profile requests.
<code>--controller-enabled</code>	Off	Starts sending controller input immediately; omit it for local inspection.

Flag	Default	Meaning
<code>-h, --help</code>	-	Prints usage and exits.

Deploy or repair the UNO Q application

Run these scripts from the project checkout on your development computer. Always supply your own SSH destination; the built-in fallback names the maintainer's board and is not appropriate for other installations.

Script or setting	Arguments and defaults	Effect
<code>scripts/deploy-uno-q-wifi.sh</code>	Optional positional <code>USER@HOST</code> ; <code>-h</code> or <code>--help</code>	Transfers the Linux application, preserves <code>data/</code> , restarts the container, and checks web routes. Does not update RetroPie or flash the matrix sketch.
<code>UNO_Q_SSH_TARGET</code>	Environment variable; overridden by a positional destination	Sets the SSH destination. Without either setting, deployment falls back to the maintainer's board.
<code>UNO_Q_APP_DIR</code>	Environment variable; default <code>/home/arduino/ArduinoApps/powerglove-vision</code>	Remote deployment directory. Changing it does not change the shutdown helper's fixed path or the machine installer's path requirement.
<code>scripts/install-uno-q-shutdown-helper.sh</code>	Optional positional <code>USER@HOST</code> ; <code>-h</code> or <code>--help</code>	Installs the fixed shutdown watcher, service, and readiness rule. Uses the positional destination, then <code>UNO_Q_SSH_TARGET</code> , then the maintainer's fallback. The application directory is fixed.

Both scripts accept at most one destination. Deployment needs existing SSH key access; helper installation can prompt for a password through the terminal.

Build, inspect, or maintain project files

These scripts run from a full development checkout. Scripts marked **no flags** do not implement `--help`; adding it is not a safe way to inspect them because they may still perform their normal work.

Script	Arguments and flags	Result or requirement
<code>scripts/build-app-lab-package.sh</code>	No flags or positional arguments	Builds <code>output/app-lab/PowerGlove-Vision-Uno-Q.zip</code> ; requires Bash, rsync, zip, and the existing public PDFs.
<code>scripts/verify-app-lab-package.py</code>	Optional <code>ARCHIVE</code> path; <code>-h, --help</code>	Checks the supplied ZIP or the default ZIP above; prints its SHA-256. Returns <code>0</code> on success, <code>1</code> on verification failure.
<code>scripts/check-documentation.py</code>	<code>--require-pdfs</code> ; <code>-h, --help</code>	Checks Markdown, links, and coverage. The optional flag also inspects the PDF set and needs <code>pypdf</code> . Returns <code>0</code> on success, <code>1</code> on failure.
<code>scripts/check-source-docs.py</code>	No flags or positional arguments	Checks source headers and docstrings; returns <code>0</code> on success or <code>1</code> on failure.

Script	Arguments and flags	Result or requirement
<code>scripts/build-docs-pdf.py</code>	No flags or positional arguments	Rebuilds all ten PDF editions; requires ReportLab. Use only when ready to regenerate the PDFs.
<code>scripts/build-gesture-crops.py</code>	No flags or positional arguments	Regenerates action illustrations from the gesture sheets; requires Pillow.
<code>scripts/fetch-runtime-assets.sh</code>	No flags or positional arguments	Downloads and verifies the pinned model into project <code>data/models/</code> ; requires Python 3 and curl.
<code>scripts/configure-uno-q-mdns.py</code>	Required positional path to the generated Compose file; no flags	Internal installer/deployment helper that edits that file. Prefer the supported setup command.
<code>scripts/profile-relay.py</code>	No flags or positional arguments	Internal unprivileged UDP relay; publishes port 55356 and forwards to the worker, with bounded packet size, outstanding requests, and timeouts.
<code>scripts/avahi-resolver-service.py</code>	No flags or positional arguments	Internal service started by the resolver brick; opens its fixed Unix socket.
<code>python/main.py</code>	No flags or positional arguments	App Lab entry point; reads <code>data/device.json</code> and supervises the worker.
<code>python/ssh_pair.py</code>	No command-line flags	Internal password-pairing helper; receives its request through standard input. Use secure Setup instead.

Understand the system commands in these guides

The table explains the subcommands and flags shown in the documentation. Defaults and additional flags for these external programs depend on their installed versions; use `man TOOL` or that tool's `--help` for its full manual.

Command or syntax	Meaning of the options used here
<code>git clone --branch dev URL</code>	Downloads a repository and checks out <code>dev</code> . A final directory argument chooses the destination folder.
<code>git branch --show-current</code>	Prints the checked-out branch name.
<code>git status --short</code>	Shows a compact list of local changes.
<code>git pull --ff-only</code>	Updates only if Git can fast-forward; refuses an automatic merge when histories have diverged.
<code>cd PATH, cd ~, pwd</code>	Changes directory, opens your home directory, or prints the current directory.
<code>command -v NAME</code>	Prints where the shell finds a command; no path means it is unavailable.
<code>sudo COMMAND</code>	Runs a command with administrator privileges. Enter your account password only at the terminal prompt.

Command or syntax	Meaning of the options used here
<code>apt update; apt install -y PACKAGES</code>	Refreshes package information, then installs packages. <code>-y</code> accepts package-manager confirmation prompts.
<code>python3 -m MODULE</code>	Runs a Python module. Used with <code>venv</code> , <code>pip</code> , <code>unittest</code> , <code>json.tool</code> , and <code>compileall</code> .
<code>python3 -m venv DIRECTORY</code>	Creates an isolated Python environment at that directory.
<code>. .venv/bin/activate</code>	Activates that environment in the current shell. The initial dot is a shell command.
<code>pip install -e PATH</code>	Installs a source checkout in editable mode. <code>[vision]</code> and <code>[receiver]</code> request optional dependency groups.
<code>pip install -U PACKAGE; --upgrade</code>	Upgrades the named package; the two options have the same meaning.
<code>unittest discover -s tests -v</code>	Finds tests in <code>tests</code> (<code>-s</code>) and prints each test's result (<code>-v</code>).
<code>compileall -q DIRECTORIES</code>	Checks/compiles Python files; <code>-q</code> reduces routine output.
<code>json.tool FILE</code>	Parses and formats JSON; a parse error means the example or configuration is invalid.
<code>PYTHONPATH=src COMMAND</code>	Sets the import path for that command only, allowing it to use the local <code>src/</code> tree.
<code>ssh-keygen -t ed25519</code>	Creates an SSH key pair using the Ed25519 algorithm (<code>-t</code>). Follow the prompts without overwriting an existing key.
<code>cat FILE</code>	Prints a text file. In key setup, print only the <code>.pub</code> public key.
<code>ssh USER@HOST</code>	Opens a remote terminal. <code>exit</code> closes that session.
<code>ssh -o BatchMode=yes USER@HOST COMMAND</code>	Runs a remote command without interactive password prompts; useful for checking key access. <code>-o</code> supplies an SSH setting.
<code>ssh -t USER@HOST COMMAND</code>	Allocates a remote terminal, allowing an interactive remote prompt.
<code>docker port CONTAINER 55356/udp</code>	Shows the host binding for the container's UDP profile port. This command does not change the container.
<code>systemctl status NAME</code>	Displays a service or timer's state. Press <code>q</code> if it opens a pager.
<code>systemctl start, stop, restart NAME</code>	Starts, stops, or restarts the named unit now.
<code>systemctl enable, disable NAME</code>	Enables or disables activation at boot. <code>--now</code> also starts or stops it immediately.
<code>systemctl is-enabled, is-active NAME</code>	Checks boot enablement or current activity.
<code>systemctl daemon-reload</code>	Reloads unit definitions after service files change.
<code>systemctl --no-block halt</code>	Requests a halt and returns without waiting. This is the helper's action, not a health check.
<code>journalctl -u NAME -n 100 --no-pager</code>	Shows logs for a unit (<code>-u</code>), limits output to the last 100 entries (<code>-n</code>), and avoids a pager.
<code>systemd-tmpfiles --create FILE</code>	Applies creation rules from the named file, such as the readiness-marker rule.
<code>grep -A8 -B2 PATTERN FILE</code>	Shows matching lines with eight following lines (<code>-A</code>) and two preceding lines (<code>-B</code>).

Command or syntax	Meaning of the options used here
<code>curl -sS URL</code>	Fetches a URL quietly (<code>-s</code>) while still showing errors (<code>-S</code>).
<code>ls -l PATH</code>	Lists files with permissions and ownership. A trailing directory slash refers to its contents.
<code>install -d -m MODE PATH</code>	Creates a directory (<code>-d</code>) with the specified permissions (<code>-m</code>). Without <code>-d</code> , <code>install</code> copies a file; <code>-o</code> sets its owner and <code>-g</code> its group.
<code>chmod MODE PATH; chown USER:GROUP PATH</code>	Changes permissions or ownership. Modes used here: <code>0600</code> owner only, <code>0640</code> owner read/write and group read, <code>0644</code> public read with owner write, <code>0700</code> private directory, <code>0755</code> publicly traversable directory or executable.
<code>rm FILE; rm -f FILE</code>	Deletes a file; <code>-f</code> suppresses prompts and ignores a missing file. Check the path before use.
<code>nano FILE</code>	Opens a text editor. Save with <code>Ctrl+O</code> , confirm the name, and exit with <code>Ctrl+X</code> .
<code>modprobe uinput</code>	Loads the Linux virtual-input kernel module.
<code>tee FILE</code>	Writes its input to a file as well as the terminal. Used with <code>sudo</code> for a protected destination.
<code>printf '%s\n' VALUE</code>	Prints a value followed by a newline.
<code>ping HOST</code>	Tests whether a host replies to network echo requests; press <code>Ctrl+C</code> to stop. It does not test the application.
<code>hostname; hostname -I</code>	Prints the host's name, or its interface addresses with <code>-I</code> .
<code>bash -n FILE; sh -n FILE</code>	Checks shell syntax without running the file.
<code>> /dev/null</code>	Discards normal output while leaving errors visible.
Pipe operator	Sends the first command's output into the second command. The vertical bar is shell syntax.
<code>\</code> at the end of a command line	Continues the same command on the next line.
<code>"\$1" through "\$4"</code>	Quoted positional arguments forwarded by RetroPie's wrapper scripts.
<code>for ...; do ...; done</code>	Repeats the enclosed commands for each value.
<code>[-r FILE]</code>	Tests whether a file is readable. <code>continue</code> skips the current loop item.
<code>\${device##*/}</code>	Extracts the last component of a path in the camera-inspection loop.

Commands shown inside a source script may use additional internal options. Those are implementation details, not extra arguments accepted by the wrapper.